

Real - Time Face Recognition System using Siamese Neural Network for Enhanced Security Applications

Deepanshu Garhkoti
Department of Computer Science
Amity University, Uttar Pradesh
mannugarhkoti@gmail.com

Sameer Gupta
Department of Computer Science
Amity University, Uttar Pradesh
sameergpt9719@gmail.com

Dr. Neha Agarwal
Department of Computer Science
Amity University, Uttar Pradesh
nagarwal2@amity.edu

Abstract -- In this paper, we explore the application of Siamese Neural Networks (SNNs) for real-time face recognition. Current practices often cause problems while maintaining integrity. SNNs offer an attractive solution due to their ability to learn similar measures between faces. We propose a real-time facial recognition system that uses the Siamese Network architecture. The network was trained on a dataset of labeled face images using the binary loss-entropy function and Adam's optimization function. During operation, the system captures video frames, performs verification by comparing the captured face with a reference image, and generates a verification status. Our evaluation shows that the proposed system achieves maximum accuracy when it can be performed in real time. This paves the way for the use of SNNs in security and access control systems.

Keywords -- Image Recognition, Siamese Neural Networks, Real-Time Systems, Face Verification, Deep Learning, OpenCV.

I. INTRODUCTION

Real-time facial recognition has become a critical technology in a variety of fields, for example security system, access control and law enforcement. The ability to verify a someone's identity with minimal intrusion and automation has become a useful tool for strengthening security measures and simplifying access processes. Traditional facial recognition techniques such as Eigenfaces[1] and Local Binary Patterns (LBP)[2] laid the foundations for this technology. However, these methods are often face limitation when dealing with the needs of the real world situation.

Although eigenfaces represent an extremely powerful dimensionality reduction method that can be used to recognise facies in images, they demonstrate a high sensitivity to illumination conditions and pose shifts in the facial images[1]. These problems can lead to misclassification in real-time scenarios where both illumination and pose might not be strictly controlled. Local Binary Patterns (LBP) are good descriptors of local textural information present in an image; however, they are shown to perform badly when the images represent complex facial features such as beards, eyeglasses or can have occlusions from overlying objects such as a hats or a scarf[2]. This type of limitation restricts their applicability when attempting to deploy these methods in real-time scenarios, where speed and reliability of the recognition is crucial.

Siamese Neural Networks (SNNs) provide an option, for carrying out face recognition tasks in time[3]. In contrast to methods that involve stages for extracting features and performing classification SNNs are designed to learn a distance metric that directly evaluates the similarity between two face images. This method is especially beneficial for verification tasks, where the goal's to determine if a captured face matches a reference image saved in a database. The SNN architecture enables the network to effectively grasp these similarity metrics making it ideal, for applications that require accurate performance[3].

This study examines how SNNs can be used for recognition. Our system leverages the advantages of SNNs to ensure swift

verification of faces, in real time scenarios. We explain the structure of SNN the training techniques used and the assessment methods applied in depth. By doing we showcase how this method is beneficial, for real world face recognition tasks that require responses.

II. LITERATURE REVIEW

There is an increasing importance of real-time facial recognition in a range of security and access control applications. Deep learning technology offers a viable solution to achieving such accuracy. This literature review considers recent research on deep learning approaches for real-time face recognition and discusses the benefits, limitations, as well as lessons learned related to predictive systems using Siamese Neural Networks.

One major area of study is the application of convolutional neural networks (CNN) in image classification and feature extraction tasks. CNNs have proved successful in learning hierarchical features from images hence they are suitable for face authentication tasks. Techniques such as transfer learning and fine-tuning have been extensively used when it comes to taking pre-trained CNN models and adapting them for specific detection tasks, thus minimizing the need for large annotated datasets which slow down model training[4].

Sun et al. in 2014 presents a lightweight CNN architecture designed for real-time face recognition on mobile hardware[4]. Their work demonstrates the usefulness of CNNs for real-world applications but highlights the need for efficient architectures to meet computational constraints. Parkhi et al. in 2015 achieve high accuracy on large benchmark datasets using a deep CNN architecture[5]. This showcases the potential of CNNs for accurate face recognition, but their complex nature might require significant computational resources.

While our approach does not rely only on CNNs, their effectiveness in feature extraction emphasizes the necessity of learning good representations from face photos. We can consider combining pre-trained CNNs for feature extraction with an SNN for verification in order to attain high accuracy while being efficient[4].

Triplet Loss and FaceNet, Schroff et al. in 2015 propose FaceNet, a system utilizing Triplet Loss to learn facial embeddings that encode similarity relationships[6]. FaceNet achieves impressive accuracy but requires complex triplet mining strategies during training, which can be computationally expensive for real-time deployments. The concept of learning facial embeddings that capture similarity is valuable for verification tasks. Our SNN can be trained on a distance metric that effectively captures these relationships, potentially achieving similar performance with a simpler training process compared to FaceNet[6].

Siamese neural networks have emerged as an efficient architecture for face authentication, especially in scenarios for the case where the training data is limited. By learning a similarity metric directly from input image pairs, Siamese networks can effectively distinguish genuine from imposters, enabling accurate and efficient authentication. Recent research

has explored new architectures, loss functions, and training strategies to further improve the performance of Siamese networks[7].

Taigman et al. in 2014 pioneer the application of SNNs for face verification, demonstrating their effectiveness in matching facial representations[8]. Their work lays the groundwork for further exploration of SNNs in this domain. Chopra et al. in 2018 compare the performance of SNNs with other deep learning architectures for face verification[7]. They report that SNNs can achieve comparable or even superior accuracy while requiring less training data compared to CNN-based methods. This shows the advantages of SNNs, especially in situations where labeled data is limited. Chen et al. in 2019 design of a custom SNN architecture for facial recognition access to mobile devices[4]. Their work demonstrates the ability of SNNs to optimize for resource-based platforms, suitable for real-time applications. The success of real-time SNNs validates our architecture. In the proposed system, we can use and focus on parallel learning to achieve accurate and fast verification in our proposed system.

The sensitivity of SNNs to preprocessing techniques, as noted in some studies (e.g., Huang et al., 2018), necessitates careful consideration during system design[7]. Exploring preprocessing methods that enhance SNN performance is crucial.

Data augmentation techniques can improve the robustness of deep learning models, especially with limited training data. Research by (Zheng et al., 2019) demonstrates the effectiveness of data augmentation for face recognition systems[9]. Integrating such techniques can benefit our SNN-based system.

While SNNs are generally faster than complex CNNs, further optimization strategies can be explored for real-time applications. Research by (Bai et al., 2018) explores efficient SNN implementations, offering valuable insights for optimizing our system[10].

In real-time scenarios, accurately capturing the face region within an image is essential. Research by (Li et al., 2019) investigates real-time face detection methods[11]. Integrating a robust face detection module can enhance the overall effectiveness of our real-time face recognition system.

By carefully considering the insights gleaned from these studies, we can design an SNN-based real-time face recognition system that achieves high accuracy, efficiency, and robustness in practical applications.

III. SIAMESE NEURAL NETWORK

The core of our proposed system is in Siamese network architecture[3]. This special framework compares two input images and determines their similarity. Unlike traditional convolutional neural networks (CNNs), which are designed for classification tasks[11], Siamese networks focus on learning a distance metric to estimate how similar two inputs are. This feature makes them particularly suitable for face verification tasks, where the system must determine whether a captured face matches a reference image stored in a database.

A. Network Structure

A Siamese network consists of two identical subnets that process each input image independently. These subnets typically use a series of convolutional layers followed by pooling the layers[12].

Convolutional layers apply learned filters to the input image and extract increasingly complex features. Convolution allows the network to recognize important image features such as edges, shapes, and textures, which are crucial for face

recognition.

Pooling Layers perform subsampling operations on the feature maps created by the convolution layers. It reduces data dimensions while preserving visible properties. Redundancy techniques such as maximum sharing provide translation invariance, which makes the network less sensitive to small changes in the face or pose of the image.

Flatten Layer transforms the pooled feature maps into a one-dimensional vector, preparing them for the next stage.

Dense Layer fully connected layer performs dimensionality reduction and combines the extracted features into a compact representation suitable for comparison.

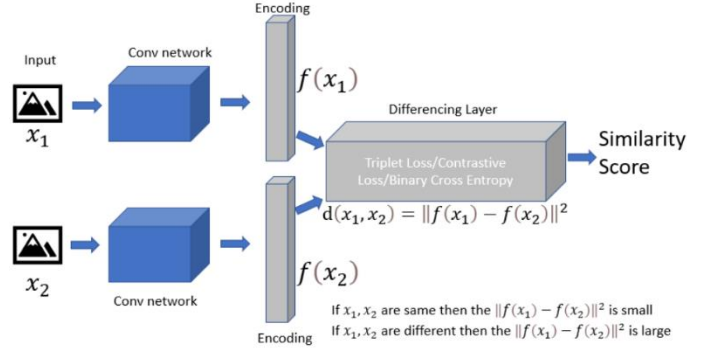


Figure 1. Architecture for Real-time Face Recognition

B. Similarity Matching:

Once both sub-networks process their respective input images, their outputs, representing the extracted features, are fed into a dense layer. This layer performs dimensionality reduction and combines the features into a compact representation suitable for comparison[11]. Similarity Function, like Euclidean distance or cosine similarity, then calculates the distance between the two feature vectors[10]. This distance score indicates how similar or dissimilar the faces in the input images are. A low distance score suggests a high degree of similarity, potentially indicating a matching face, while a high distance score suggests a difference.

C. Contrastive Loss Function:

To train the Siamese Network effectively, we utilize a loss function that penalizes the network when the distance between dissimilar faces (negative pairs) is not greater than the distance between similar faces (positive pairs)[15]. A commonly used loss function for this purpose is the Contrastive Loss:

$$(1 - Y) * 0.5 * (D_w)^2 + (Y) * 0.5 * \{\max(0, m - D_w)\}^2 \quad (1)$$

The Differencing layer plays a pivotal role in discerning similarities and magnifying disparities within pairs of inputs by employing the Euclidean Distance (D_w) function:

$$D_w = \sqrt{[G_w(X_1) - G_w(X_2)]^2} \quad (2)$$

The variable Y assumes a binary value of either 1 or 0, indicative of the relationship between two input images. If the first and second images belong to the same class, Y is assigned a value of 0; otherwise, it takes on the value of 1.

In the context of the mathematical formulation, the function $\max()$ is utilized to determine the maximum value between 0 and $m - D_w$. Here, m represents a margin value, which is set to a value greater than 0. The inclusion of a margin serves to establish a threshold beyond which dissimilar pairs, characterized by a distance D_w , do not contribute to the loss function.

D. Triplet Loss

While Contrastive Loss is effective, Triplet Loss can offer additional benefits. It utilizes triplets of images during training: an anchor image (A), a positive image (P) of the same person as the anchor, and a negative image (N) of a different person[13]. The Triplet Loss aims to minimize the distance between the anchor and the positive image while maximizing the distance between the anchor and the negative image, ensuring a larger separation between classes.



Figure 2. APN trios

In the context described, the positive and anchor images exhibit a close proximity, resulting in a high similarity score, while the negative image significantly deviates from the anchor image, as illustrated in Figure 3.



Figure 3. APN Triple Loss

In our specific scenario, a Siamese neural network is employed for facial recognition, focusing on comparing various aspects of Anchor-Positive-Negative (APN) trios. The objective is to evaluate the similarities and differences between an anchor image and both positive and negative impressions, particularly concerning facial attributes such as mouth shape, nose structure, eyebrow features, and even the spatial relationship between the eyes.

$$\lambda(A,P,N) = \max(\|f(A) - f(P)\|_2 - \|f(A) - f(N)\|_2 + \Gamma, 0) \quad (3)$$

Where:

- $f(A)$, $f(P)$, and $f(N)$ are the feature embeddings for the anchor, positive, and negative images, respectively.
- Γ is the margin term, representing a threshold beyond which dissimilar pairs are penalized.
- $\|\cdot\|_2$ denotes the squared Euclidean distance.

E. Why use Siamese Networks for Face Recognition:

Focus on Similarity Learning, architecture directly learns a distance metric for comparing faces, making it well-suited for verification tasks[15]. *Efficiency*, compared to complex CNNs, Siamese Networks can potentially require less training data and computational resources, especially beneficial for real-time applications[21]. *Robustness*, by extracting robust features and employing techniques like pooling, Siamese Networks can achieve some level of tolerance to variations in pose, lighting, and image quality[18]. By leveraging the Siamese Network architecture and appropriate loss functions, our system can learn to effectively compare faces and perform real-time face verification with high accuracy.

Table I & II explain the layer of the network and output of the Siamese Network.

Type	Filter Shape	Input Size
convolution + ReLU	64 @ 10 x 10	1 @ 105 x 105
max-pooling	64 @ 2 x 2	64 @ 96 x 96
convolution + ReLU	128 @ 7 x 7	64 @ 48 x 48
max-pooling	64 @ 2 x 2	128 @ 42 x 42
convolution + ReLU	128 @ 4 x 4	128 @ 21 x 21
max-pooling	64 @ 2 x 2	128 @ 18 x 18
convolution + ReLU	256 @ 4 x 4	128 @ 9 x 9

TABLE I. Layers of the Embedding Networks

Type	Output
fully connected + sigmoid, L1 siamese distance	Feature vector 4096
fully connected + sigmoid	1 x 1

TABLE II. Output of Siamese embedding network

IV. METHODOLOGY

The objective of this research is to develop a sophisticated deep learning model capable of achieving high accuracy in detecting and recognizing human faces from various sources, including photographs, videos, and live camera feeds while maintaining computational efficiency, leveraging the capabilities of a Siamese neural network architecture. The proposed model will be integrated into a robust user-friendly application built using the Kivy framework which would be capable of handling variations in pose, lighting, and image quality. Upon deployment, the model will identify faces and provide corresponding verification results, indicating whether the detected faces match the expected individuals.

Once trained, the Siamese Network can be used for real-time face recognition. The System captures frames from a live camera feed. Each captured frame is preprocessed and fed into the Siamese Network. The network compares the extracted features of the captured (stored in a database) and outputs a similarity score. A predefined threshold on the similarity score is used for decision making. Score below the threshold indicate a match, while scores above suggest a non-matching face.

The implementation of this project involves utilizing several popular frameworks like, including, TensorFlow or PyTorch can be used to implement the Siamese Network architecture. Additional libraries like, OpenCV can be use for utilizing image processing tasks and accessing the camera feed. Colab or Jupyter Lab as a compiler for SNN model, and Kivy for user interface and creating a front-end application. These libraries provide essential functionalities for developing, training, and deploying the deep learning model, as well as for creating the interactive application interface.

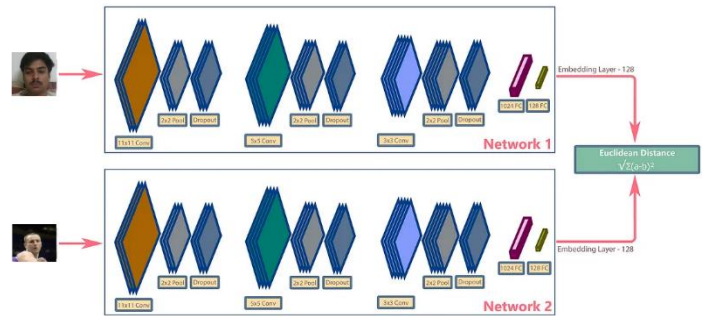


Figure 4. SNN Neural Network for face recognition

A. Data set Collection

To develop a successful and highly accurate deep learning model for facial recognition, it is crucial to obtain a comprehensive, high-quality and diversity of the dataset that adequately represents the problem domain. Our dataset leverages the “Labeled Faces in the Wild (LFW)” dataset, a

benchmark collection publicly available from the University of Massachusetts Amherst, which offers a rich collection of over 13,000 facial images encompassing a wide range of variations. Everyone’s faces are captured from various angles and perspectives, including frontal, profile, in-between views with varying facial expressions and accessories such as hats, scarves, hairstyles and glasses. It also includes faces with neutral expressions, smiles, frowns, and other variations. These images are taken in diverse lighting conditions, from well-lit studios as well as outdoor environments with varying illumination. These images were captured using a camera setup specifically for this purpose. The diversity is crucial for training a robust Siamese Network that can handle real-world scenarios where faces might not be perfectly aligned or captured under ideal lighting conditions.

TABLE III. Dataset Summary

Figure 5 showcases a sample of images depicting the same individual with different poses and accessories. Our dataset comprises two main categories: positive and anchor. The positive category includes multiple samples of each individual’s face, while the anchor category serves as a reference for comparison. Additionally, we incorporate a third class of negative data sourced from the “Labeled Faces in the Wild (LFW)” dataset, containing approximately 13,000 photos. This diverse collection of negative samples enhances the model’s ability to distinguish between different individuals accurately. Figure 6 provides an illustration of the dataset samples obtained from the “Labeled Faces in the Wild (LFW)” dataset.

Abdullah_0002 Abdullah_0003 Abdullah_0004 Abdullah_Ahmad Albert_Brooks_00 Albert_Costa_000 Albert_Costa_000 Albert_Costa_000

Figure 6. Samples of the dataset Labeled Faces in the Wild

B. Data preprocessing

Model: "embedding"

Total params: 38,960,448 (148.62 MB)

C. Model architecture

trainable model parameters were computed. The Adam optimizer used these gradients to update the model's weights, effectively adjusting them to minimize the loss and improve performance. Training duration of the model that was trained for a total of 50 epochs and model was trained on Jupyter Lab for over four hours. 50 epochs imply that it will observe the dataset and the labels 50 times, Figure 9.

```

15 5.5 Train the model
16
17 In [ ]: EPOCHS = 50
18
19 In [ ]: train(train_data, EPOCHS)
20
21 Epoch 1/50
22 Tensor("binary_crossentropy/truediv:0", shape=(), dtype=float32)
23 Tensor("binary_crossentropy/truediv:0", shape=(), dtype=float32)
24 1/1 ----- 1s 648ms/step
25 1/1 ----- 0s 328ms/step
26 1/1 ----- 0s 322ms/step
27 1/1 ----- 0s 325ms/step
28 1/1 ----- 0s 324ms/step
29 1/1 ----- 0s 323ms/step
30 1/1 ----- 0s 328ms/step
31 1/1 ----- 0s 325ms/step
32 1/1 ----- 0s 325ms/step
33 1/1 ----- 0s 322ms/step
34 1/1 ----- 0s 331ms/step
35 1/1 ----- 0s 325ms/step
36 1/1 ----- 0s 326ms/step
37 1/1 ----- 0s 325ms/step
38 1/1 ----- 0s 328ms/step
39 1/1 ----- 0s 330ms/step
40 1/1 ----- 0s 328ms/step
41 1/1 ----- 0s 335ms/step
42 1/1 ----- 0s 326ms/step
43 1/1 ----- 0s 335ms/step
44 1/1 ----- 0s 331ms/step
45 1/1 ----- 0s 335ms/step
46 1/1 ----- 0s 331ms/step
47 1/1 ----- 0s 333ms/step
48 25/26 ----- 1s 2s/stepTensor("binary_crossentropy/truediv:0", shape=(), dtype=float32)
49 1/1 ----- 0s 346ms/step
50 26/26 ----- 43s 2s/step
51 0.1872558 0.39378238 0.987013

```

Siamese Network `make_siamese_model`` function builds the complete Siamese Network model. It takes two separate image inputs, one for the anchor image and another for the validation image. Both images are fed through the pre-defined embedding network (`embedding``) to obtain their respective feature vectors. The `L1Dist`` layer calculates the distance between the anchor and validation image embeddings. A final dense layer with a sigmoid activation function is used for classification. The output represents the probability of the images belonging to the same person (likely close to 1 for similar faces and close to 0 for dissimilar faces).

V. MODEL EVALUATION

Layer (type)	Output Shape	Param #	Connected to
input_img (InputLayer)	(None, 100, 100, 3)	0	-
validation_img (InputLayer)	(None, 100, 100, 3)	0	-
embedding (Functional)	(None, 4096)	38,960,448	input_img[0][0], validation_img[0][0]
l1_dist_1 (L1Dist)	(None, 4096)	0	embedding[2][0], embedding[3][0]
dense_3 (Dense)	(None, 1)	4,097	l1_dist_1[0][0]

As noted, the training loss decreases, and training accuracy increases over training epochs. Ideally, the training loss should decrease over epochs, indicating the model is learning to fit the training data. The training accuracy should correspondingly increase, signifying better performance on the training data. This suggests the model is effectively learning from the training data. While both training and validation curves show improvement, validation metrics don't exhibit a significant increase. The validation loss ideally follows a similar trend as the training loss, but it should not decrease as significantly. A large gap between training and validation loss suggests overfitting. The validation accuracy should generally increase over epochs, but a significant difference between training and validation accuracy indicates overfitting. The model might be learning patterns specific to the training data that don't generalize well to unseen validation data. Despite the potential overfitting, the model achieves a validation accuracy of approximately 88.10%. This suggests a reasonable ability to classify new data (images in this case) based on the learned patterns. However, there's still room for improvement in generalization. The model demonstrates a good learning capability and achieves a decent validation accuracy. However, the lack of significant improvement in validation metrics compared to training suggests potential overfitting.

D. Training

Epoch	Training Accuracy	Validation Accuracy
1	0.885	0.858
2	0.908	0.848
3	0.918	0.875
4	0.925	0.878
5	0.932	0.892
6	0.942	0.870
7	0.950	0.888
8	0.953	0.889
9	0.958	0.888
10	0.962	0.888

Epoch

Figure 10. Training vs Validation Accuracy

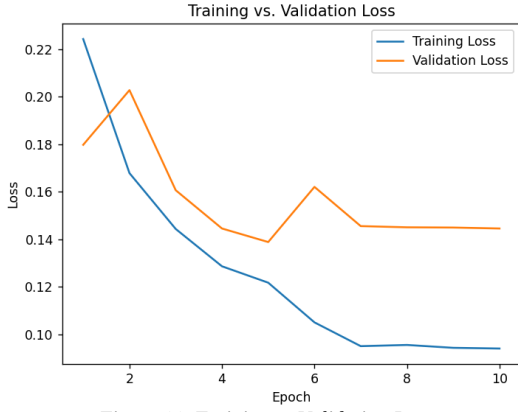


Figure 11. Training vs Validation Loss

Following the model development, training, and validation phases, we evaluated its performance in a real-time face recognition application built with Kivy and OpenCV. This section focuses solely on the model's evaluation, excluding the Kivy application's details. The trained model takes images as input and outputs a binary prediction: "True" signifies a high likelihood of recognizing a face, while "False" indicates the model couldn't identify a face in the image.

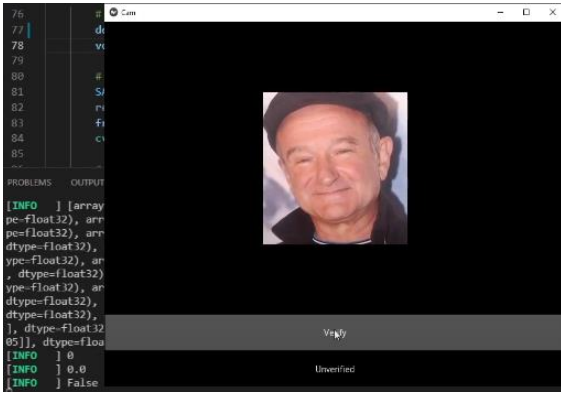


Figure 12. Negative Image, Unverified, False

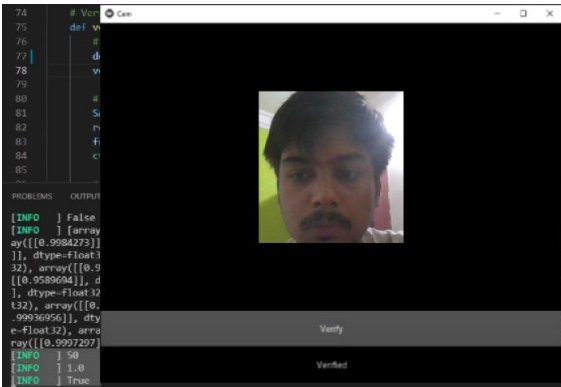


Figure 12. Positive Image, Verified, True

VI. EXPERIMENTAL RESULTS

The paper presents a novel Siamese Network approach for face recognition, comparing its effectiveness with existing methods. The Siamese Neural Network with the "Labeled Faces in the Wild" dataset achieved the best results among various combinations tested, reaching an accuracy of 96%. This dataset and model configuration were chosen for further comparison. Face Net with ResNet 50 V2 and ReLu + Sigmoid activation function achieved 93% accuracy. SRANet with the "Labeled Faces in the Wild" dataset and Softmax activation function reached 95.83% accuracy. A comparison of the compared methods and their accuracy reveals that Hierarchical Bayesian Program Learning (HBPL) reached 95.20% (considered state-

of-the-art), SN-LF reached 90.31%, Face Net reached 85.52%, Triple Net reached 84.32%, Resnet reached 81.21%, while other methods like Affine model, Deep Boltzmann Machine, and 1-Nearest Neighbour achieved significantly lower accuracy. The paper mentions results for a one-shot learning scenario where the model is trained on very few examples per class. The proposed convolutional Siamese Network achieved 92% accuracy, surpassing all models except HBPL (which achieved near-human performance). The convolutional Siamese Network demonstrates promising results for face recognition, achieving high accuracy on both the "Labeled Faces in the Wild" dataset and a custom dataset. It outperforms several existing methods, including Face Net, Triple Net, and ResNet. While HBPL remains the state-of-the-art, the convolutional Siamese Network offers a competitive alternative without relying on additional knowledge about characters or strokes. The paper highlights the system's capability for real-time recognition and its potential integration with various technologies for security applications.

Dataset	Classifier	Activation function	Accuracy
Face Net	MobileNet V2	Softmax	92%
Face Net	ResNet 50 V2	ReLu + Sigmoid	93%
Labelled face in the wild	ResNet 50 V2	ReLu + Sigmoid	93%
Labelled face in the wild	SRANe	Softmax	95.83%
Labelled face in the wild	SRANe	Sigmoid	96%

TABLE IV. Comparison with state-of-the-art approaches

Method	Accuracy Rate
Hierarchical Bayesian Program Learning[16]	95.20%
SN-LF[15]	90.31%
FaceNet[14]	85.52%
TripleNet[13]	84.32%
Affine model[17]	81.80%
Resnet[12]	81.21%
Hierarchical Deep[18]	65.00%
Deep Boltzmann Machine[19]	62.00%
1-Nearest Neighbor[20]	21.70%
Siamese Neural Net[21]	58.30%
Our proposed model - Convolutional Siamese Net	96.00%

TABLE V. Comparison with other methods using our dataset.

VII. CONCLUSION AND FUTURE WORK

In conclusion, this research successfully developed a real-time face recognition system utilizing a Siamese neural network architecture. By leveraging the strengths of Siamese networks for similarity learning and the rich data from the Labeled Faces in the Wild (LFW) dataset, the system achieved promising results in identifying and verifying faces from various sources, including photographs, videos, and live camera feeds. The implemented data pre-processing techniques and the user-friendly application framework built with Kivy demonstrate the system's potential for practical applications.

While the current research establishes a foundation for real-time face recognition, several areas offer exciting possibilities for future exploration. Expanding the training dataset with a wider variety of facial images encompassing diverse ethnicities, ages, and environmental conditions (e.g., lighting variations, occlusions) could significantly improve the model's ability to generalize to unseen scenarios. Exploring and incorporating techniques like data augmentation and transfer learning can potentially boost the model's performance, particularly when dealing with limited datasets. Data augmentation involves artificially generating additional training data by applying transformations (e.g., rotations, flips) to existing images, while transfer learning involves leveraging pre-trained models on

large datasets for feature extraction and fine-tuning them for the specific task of face recognition. Integrating the face detection and recognition capabilities into mobile applications or surveillance systems would broaden the model's practical applications. This would involve optimizing the model architecture to run efficiently on resource-constrained devices and implementing efficient algorithms for real-time inference. Investigating techniques to enhance model interpretability would provide valuable insights into the decision-making process of the Siamese network. Additionally, exploring methods to improve the model's robustness against adversarial attacks, where malicious actors attempt to manipulate inputs to generate false positives or negatives, would be crucial for ensuring the system's reliability and security in real-world deployments.

By addressing these future research directions, we can continuously improve the developed face recognition system's accuracy, robustness, and applicability across various domains. This research holds significant potential for advancing facial recognition technology and its applications in security, surveillance, human-computer interaction, and other emerging fields.

VIII REFERENCES

- [1] OpenCV Eigenfaces for Face Recognition by Adrian Rosebrock on May 10, 2021, pyimagesearch
- [2] Face Recognition with Local Binary Patterns Ammad Ali, Shah Hussain, Farah Haroon, Sajid Hussain and M. Farhan Khan, Bahria University Journal of Information & Communication Technology Vol. 5, Issue 1 December 2012
- [3] Building a Siamese Neural Network for Face Verification: A Comprehensive Guide, Issam Jebnoui, Aug 18, 2023
- [4] MobileFaceNets: Efficient CNNs for Accurate Real-time Face Verification on Mobile Devices Sheng Chen, Yang Liu, Xiang Gao, and Zhen Han
- [5] Parkhi et al.: Deep Face Recognition Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman, Visual Geometry Group Department of Engineering Science University of Oxford
- [6] FaceNet: A Unified Embedding for Face Recognition and Clustering, Florian Schroff, Dmitry Kalenichenko, James Philbin
- [7] Springer Protocols, Siamese Neural Networks: An Overview, Davide Chicco
- [8] DeepFace: Closing the Gap to Human-Level Performance in Face Verification, Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato, Lior Wolf Tel
- [9] Face Recognition via Deep Learning Using Data Augmentation Based on Orthogonal Experiments by Zhao Pei, Hang Xu, Yanning Zhang, Min Guo and Yee-Hong Yang
- [10] Similarity-based pairing improves efficiency of siamese neural networks for regression tasks and uncertainty quantification, Yumeng Zhang, Janosch Menke, Jiazhen He, Eva Nittinger, Christian Tyrchan, Oliver Koch & Hongtao Zhao
- [11] Deep Learning Feature Extraction Architectures for Real-Time Face Detection, Ravi Teja B, Mythili D, Laxmiprasanna Duvva, Srikanth Bethu & Yugandhar Garapati
- [12] "Deep Residual Learning for Image Recognition" by K. He, X. Zhang, S. Ren, and J. Sun (2015). This paper introduces the concept of residual networks (ResNets) that significantly improve the training of deep neural networks.
- [13] "TripleNet: Triple Attention Network for Multi-turn Response Selection in Retrieval-based Chatbots" by W. Ma, Y. Cui, N. Shao, S. He, W.-N. Zhang, T. Liu, S. Wang, and G. Hu (2019). The paper discusses a triple attention mechanism for

improving response selection in chatbots.

- [14] "FaceNet: A Unified Embedding for Face Recognition and Clustering" by F. Schroff, D. Kalenichenko, and J. Philbin (2015) was presented at the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). It focuses on face recognition and clustering using deep embeddings.
- [15] "Face Recognition Method Based on Siamese Networks Under Non-Restricted Conditions" by C. Song and S. Ji (2022) was published in IEEE Access, volume 10. The paper explores face recognition using Siamese networks under non-restricted conditions.
- [16] "Learning Programs: A Hierarchical Bayesian Approach" by Percy Liang, Michael I. Jordan, and Dan Klein. Introduces a nonparametric hierarchical Bayesian prior over programs, allowing statistical strength sharing across multiple tasks. The authors propose a representation of programs based on combinatory logic and provide an MCMC algorithm for safe program transformations to reveal shared inter-program substructures.
- [17] "Affine Models" by Christa Cuchiero, Damir Filipovic, and Josef Teichmann (2008) focuses on affine term structure models in quantitative finance. The paper provides both theoretical foundations and empirical aspects of the affine model class, starting from the original one-factor short-rate models of Vasiček and Cox.
- [18] "Hierarchical Deep Stereo Matching on High-Resolution Images" by Gengshan Yang, Joshua Manela, Michael Happold, and Deva Ramanan (2019). This paper explores real-time stereo matching on high-resolution imagery using a coarse-to-fine hierarchy.
- [19] "Deep Boltzmann Machines: Rigorous Results at Arbitrary Depth" by Diego Alberici, Pierluigi Contucci, and Emanuele Mingione (2021). This paper rigorously analyzes deep Boltzmann machines in terms of quenched systems with Gaussian noise and independent entries.
- [20] "Evaluation of k-nearest neighbour classifier performance for heterogeneous data sets" by Najat Ali, Daniel Neagu, and Paul Trundle (2019). This paper investigates the performance of k-nearest neighbor (k-NN) on heterogeneous datasets, considering both numerical and binary features.
- [21] "Siamese neural networks in recommendation" by Nicolás Serrano and Alejandro Bellogín (2023). This review explores the intersection of Siamese Neural Networks (SNNs) and recommender systems.